

Security Audit

Tand3m V3 (Launchpad)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	23
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Tand3m team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Tand3m is a cutting-edge launchpad platform at the intersection of Web3 and artificial intelligence, designed to revolutionize the way users interact with decentralized technologies. By seamlessly integrating AI capabilities with blockchain infrastructure, Tand3m empowers users to create, manage, and optimize decentralized applications (dApps) and smart contracts with unprecedented ease and efficiency. The platform offers intuitive tools and resources that cater to both seasoned developers and newcomers, facilitating the adoption and implementation of Web3 solutions across various industries. With a focus on user-centric design and robust security measures, Tand3m is poised to accelerate the mainstream acceptance of decentralized technologies, driving innovation and fostering a more inclusive digital economy.

Project Name: TAND3M V3

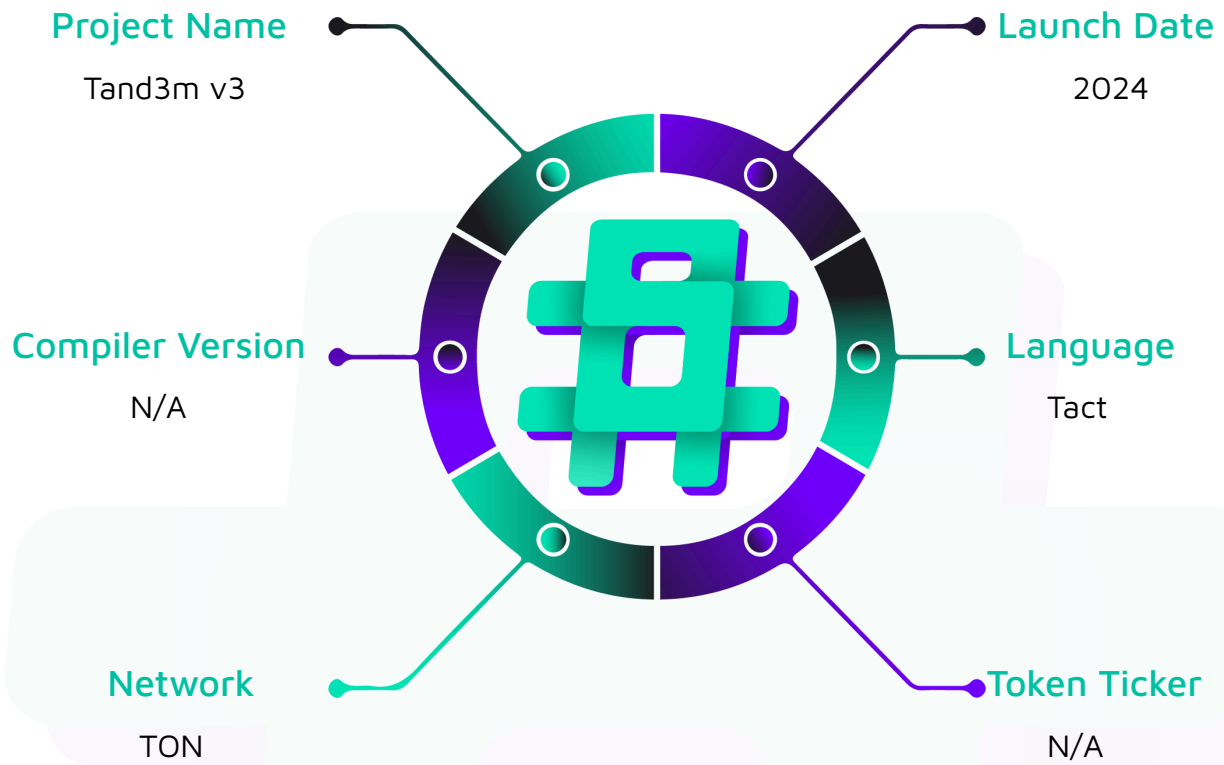
Project Type: Launchpad

Compiler Version: N/A

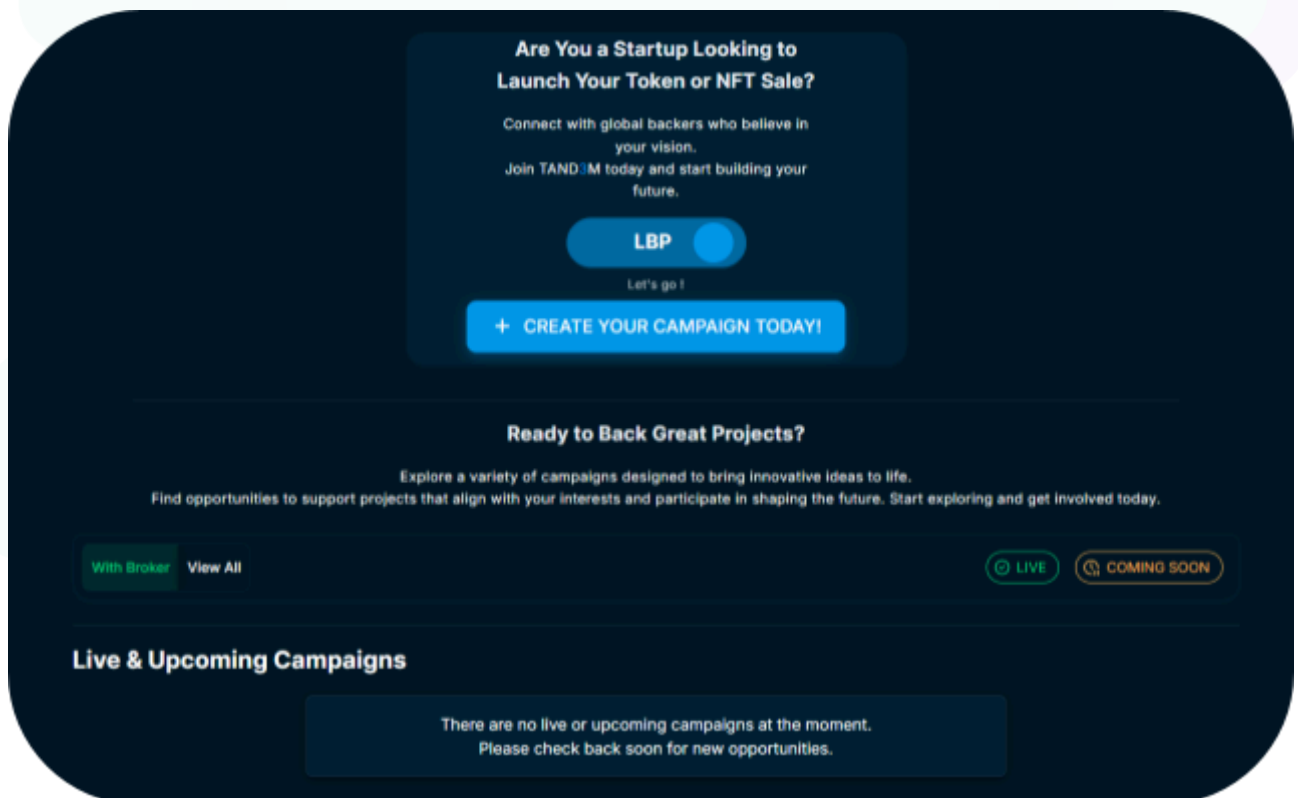
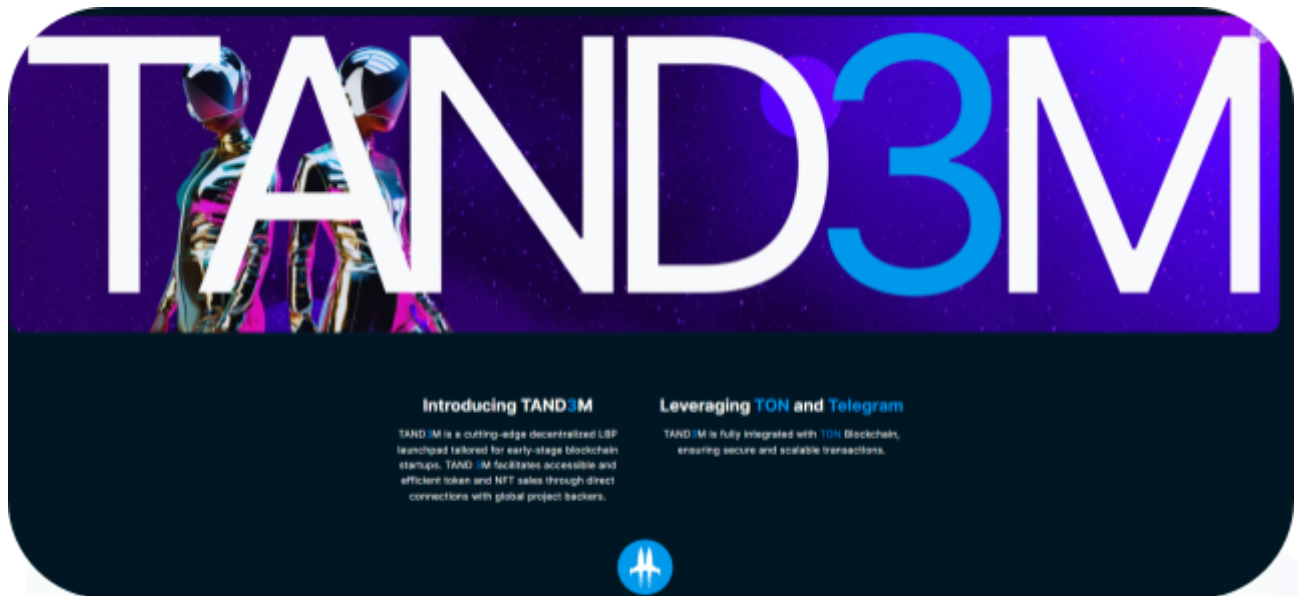
Website: tand3m.io

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the tact code within the Tand3m project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing. The audited source code files correspond to the code cell hex hash below:

Description	TAND3M Protocol Smart Contracts
Platform	TON / Tact
Audit Date	January, 2025
Contract 1	marketplace.tact
Contract 2	coin_sale.tact
Contract 3	shared.tact
Contract 4	messages.tact
Contract 5	holder.tact
Contract 6	name.tact
Contract 7	lbp.tact
Contract 8	nft_sale.tact
Code Cell Hash	7f833edce5365f0467f32e38c83c34d6188d0f077ee89 3a549d6b229d4eb61f0

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. We initially identified some vulnerabilities that have since been addressed by Tand3m.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

The Tand3m team has resolved all vulnerabilities initially identified below.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

2 Low severity vulnerabilities

4 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Marketplace.tact - Main contract handling deployment and management of sales, fee configurations, and administrative functions	Contract achieves this functionality.
CoinSale.tact - Implements liquidity pool functionality for token sales with configurable weight decay parameters	Contract achieves this functionality.
NftSale.tact - Manages NFT marketplace operations including listing, buying, and price management	Contract achieves this functionality.
Shared.tact - Library contract containing shared structs, messages and constants used across other contracts	Contract achieves this functionality.
Messages.tact - Library contract defining message structures for inter-contract communication	Contract achieves this functionality.
Holder.tact - Library trait implementing Jetton (token) transfer functionality	Contract achieves this functionality.
Name.tact - Simple library trait providing naming functionality	Contract achieves this functionality.
LBP.tact - Library trait implementing Liquidity Bootstrapping Pool calculations and weight management	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Tand3m project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Tand3m project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `Nft_sale.tact` - Missing sender validation in NFT ownership assignment handler enables unauthorized NFT acceptance

Description

The `receive(OwnershipAssigned)` function in `Nft_sale.tact` lacks sender validation, allowing any address to create fake NFT listings. A malicious contract can craft an `OwnershipAssigned` message with a legitimate `prev_owner_a` value (e.g., the marketplace's `tee_a` address) and valid gas amount, bypassing the only safety check `msg.prev_owner_a != self.tee_a`. This creates a "ghost listing" where the contract thinks it has an NFT that was never actually transferred.

Vulnerability Details

The vulnerability stems from the function adding entries to `self.items` mapping without authenticating the NFT transfer. A malicious contract can craft the `OwnershipAssigned` message with specific parameters to impersonate legitimate NFT transfers. When such a message passes the gas check and `prev_owner` validation, the contract registers the sender in the items mapping, creating a listing for an NFT that doesn't exist.

```
receive( _msg: OwnershipAssigned) {  
    require( context().value >= NFT_MIN_M, "Not enough gas");  
  
    if( _msg.prev_owner_a != self.tee_a) {  
        self.transfer_item( sender(), _msg.query_id, _msg.prev_owner_a);  
  
        self.send_excesses( _msg.prev_owner_a);  
  
        return;  
    }  
}
```

```

let item_price= self.default_price;

try {

    if( ! _msg.forward_payload.empty()) {

        item_price= ItemX.fromSlice( _msg.forward_payload).price;

    }

} catch( err) {

    self.transfer_item( sender(), _msg.query_id, _msg.prev_owner_a);

    self.send_excesses( _msg.prev_owner_a);

    return;

}

self.items.set( sender(), ItemX2 {

    price: item_price,

    sold_at_st: 0,

});

self.send_excesses( _msg.prev_owner_a);

}

```

As per the documentation (<https://docs.tact-lang.org/cookbook/nfts/#accepting-nft-ownership-assignment>), this operation should have access control added.

Impact

Attackers can create fake listings for non-existent NFTs, tricking users into sending TON for phantom assets.

Recommendation

Store the expected NFT collection address and validate that the sender matches this address before accepting the transfer.

Status

Resolved

Medium

[M-01] Nft_sale.tact - Missing access control allows unauthorized NFT recovery

Description

In `Nft_sale` contract, the `ReturnStuckItem` message handler allows any user to transfer NFTs to the admin address if the NFT is not registered in the items mapping.

Vulnerability Details

While registered but unsold items require admin rights for transfer, and sold items can only be transferred to their rightful buyers, there is no access control on the first condition.

This means that any NFT mistakenly sent to the contract that hasn't been properly registered can be forcefully transferred to the admin address by any user, potentially resulting in loss of NFTs that were intended for future sales or mistakenly sent.

```
receive( _msg: ReturnStuckItem) {  
  
    require( context().value >= NFT_MIN_M, "Not enough gas");  
  
    let x= self.items.get( msg.itema);  
  
    if( x == null) {  
  
        self.transfer_item( msg.itema, msg.queryid, self.tee_a);  
  
    } else if( x!!.sold_at_st == 0) {  
  
        self.require_admin_rights();  
  
        self.transfer_item( msg.itema, msg.queryid, self.tee_a);  
  
    } else {  
  
        self.transfer_item( msg.itema, msg.queryid, x!!.buyer_a!!);  
  
    }  
}
```

```
}  
  
self.send_excesses( sender());  
  
}
```

Impact

Any unregistered NFT in the contract can be forcefully transferred to the admin address by malicious users, leading to potential loss of assets and disruption of planned sales.

Recommendation

Add admin rights check for all NFT recovery paths in ReturnStuckItem handler, not just for registered unsold items.

Status

Resolved

Low

[L-01] Coin-sale.tact - Missing slippage protection

Description

The `receive(Swap)` function in `Coin-sale.tact` lacks slippage protection, which is a safety mechanism for any swap functionality. The function calculates the output amount based on the current state but does not verify if this amount meets any minimum output requirement specified by the user.

The code only validates the input amount and contract state, then proceeds with the swap using the calculated output amount without any checks against potential slippage.

Vulnerability Details

When a user submits a swap transaction, there is a delay between transaction submission and execution. During this period, other transactions can be executed that change the pool's state and thus the effective exchange rate. The current

implementation in `receive(Swap)` calculates `out_m` based on either the fixed price (`fp`) or the dynamic calculation via `calc_out_m()`, but never validates if this output amount is acceptable to the user. This leaves users vulnerable to front-running attacks where malicious actors can manipulate the exchange rate by submitting their own transactions that execute before the user's transaction.

Impact

Users may receive significantly fewer tokens than expected due to front-running attacks or high market volatility, leading to direct financial losses.

Recommendation

Add a minimum output amount parameter to the Swap message and validate that the calculated output meets this minimum requirement before executing the swap.

Note

The Hashlock team initially categorized this finding as a medium vulnerability due to the possibilities of slippage to scale up in the future. After discussing with the development team from Tand3m, the finding was moved to Low severity since it is a design decision and MEV is not a concern on the TON network.

Status

Acknowledged

[L-02] Sale contracts - Excess funds returned to admin instead of to sender

Description

In multiple contracts including `Nft_sale.tact` (lines 126, 217, 225) and `Coin_sale.tact`, excess funds are consistently returned to the admin address (`tee_a`) instead of the transaction sender.

This is evident in various functions such as `Deploy` and `UpdateCfg` handlers where `send_excesses()` forwards remaining gas to `tee_a`.

Since the admin role can be assigned to multiple addresses (`mp_a` and `tee_a` as seen in `require_admin_rights()`), returning funds to a fixed address causes unnecessary gas consumption for additional message forwarding if called by another user.

Recommendation

Modify `send_excesses()` calls to return funds to the original `sender()` instead of `tee_a`.

Status

Resolved

QA

[Q-01] Marketplace.tact - Commented out potentially insecure code

Description

In `Marketplace.tact`, there are several blocks of commented-out code at lines 106-110, 153-160, and 207-210 that implement a registration system for deployed sales. The implementation includes a counter limit and a mapping of registered sales addresses.

If this system were to be implemented, it could be vulnerable to denial of service attacks since registration appears to be free and could allow an attacker to exhaust the registration limit with no cost barrier.

Recommendation

Remove the commented code blocks to prevent future reuse of the potentially vulnerable implementation and consider implementing a more robust registration system if needed.

Status

Resolved

[Q-02] Nft_sale.tact - Empty receive message

Description

In `Nft_sale.tact`, lines 242-244 contain a receive handler that accepts any message (`_msg: Slice`) but doesn't process the input data. Instead, it unconditionally forwards excess funds to the marketplace address (`mp_a`). This could lead to confusion as there's no clear purpose for accepting arbitrary messages, or simply the attached funds may be lost.

Recommendation

Either implement proper message handling logic or remove the empty receive handler if this functionality is not needed.

Status

Resolved

[Q-03] Nft_sale.tact - Redundant minimum value check

Description

In `Nft_sale.tact`, there are two consecutive minimum value checks performed at lines 172 and 175. The first check verifies if `ctx.value` is greater than or equal to `NFT_MIN_M`, while the second check verifies if `ctx.value` is greater than or equal to `x.price + NFT_MIN_M`. Since the second check is strictly larger than the first one due to the addition of `x.price`, the first check becomes redundant.

Recommendation

Perform the merged check already in the earlier line (172).

Status

Resolved

[Q-04] Sale contracts - Marketplace admin retains privileged access to all deployed sales

Description

In both `Nft_sale` and `Coin_sale` contracts, the `require_admin_rights()` function grants administrative privileges to both the marketplace address (`mp_a`) and the contract admin (`tee_a`) through the condition `sender() == self.mp_a || sender() == self.tee_a`.

This means the marketplace deployer maintains permanent administrative access to all deployed sale contracts, including withdrawal capabilities. If the marketplace admin address is compromised, an attacker could potentially drain funds from all active sale contracts in the system.

Recommendation

Consider removing marketplace admin (`mp_a`) from the admin rights check after successful contract deployment to reduce the privileged access surface area.

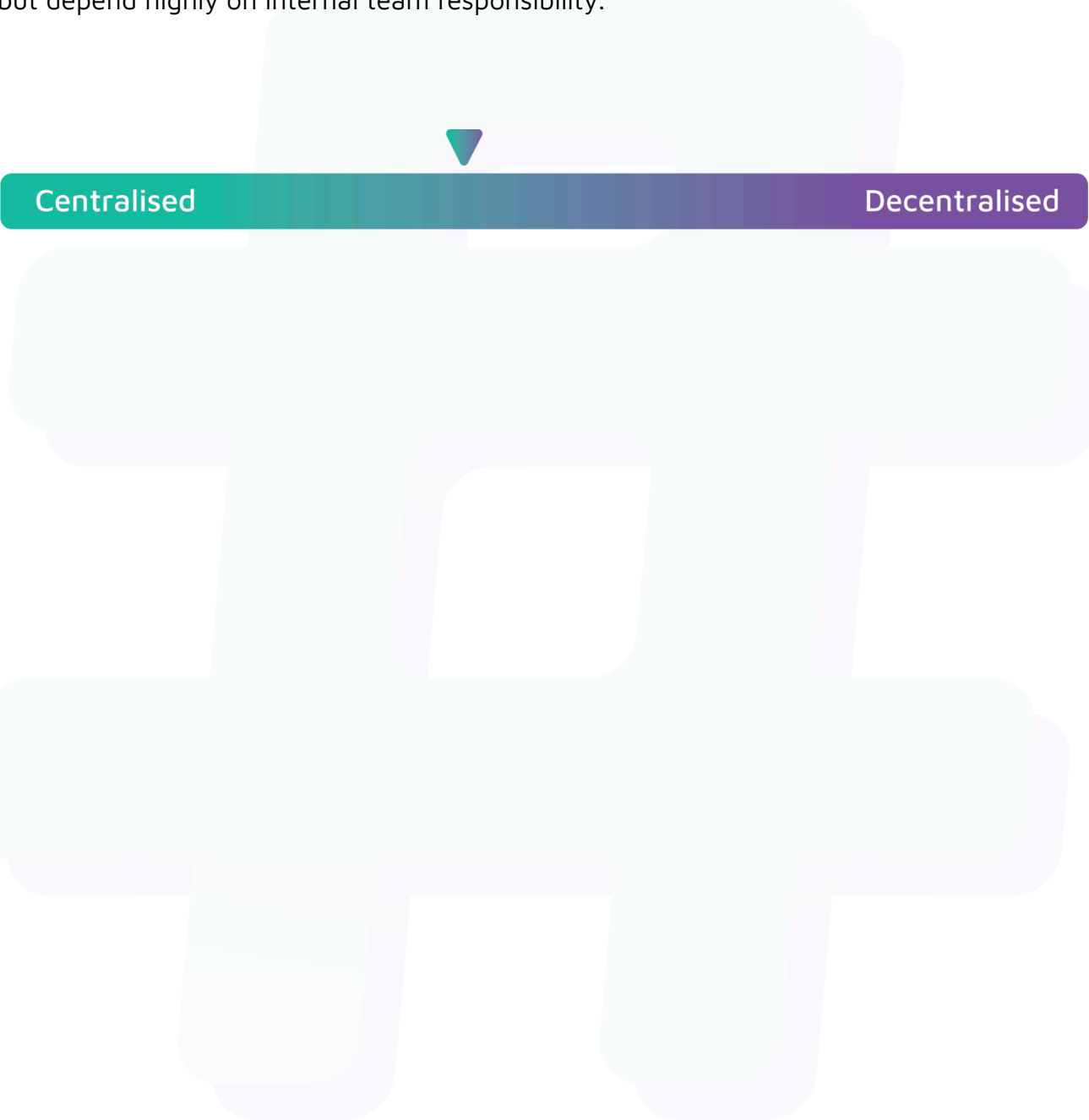
Status

Acknowledged

Centralisation

The Tand3m project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Tand3m v3 project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.